

DataSenado Lake Aberto

Notas construtivas de arquitetura —
em resposta ao feedback do time

Natureza	Complemento à revisão cética de 27/ago: toma o objetivo declarado como dado e projeta a partir dele. Abre com o acerto de contas — o que se retira, o que se mantém.
Objetivo assumido	Interface analítica e conversacional no browser, via HTTPS range, majoritariamente sobre assets estáticos governados por CDN, com backend serverless minimalista para agregações intensas, caches e modelos conversacionais do Senado; WebMCP para os modelos do próprio usuário.
Data	27 de agosto de 2026

Acerto de contas com o feedback

Ponto do time	Posição revista
"workerd e miniflare são open source"	Retirado. Correto: <code>workerd</code> é Apache-2.0, <code>miniflare</code> idem, Durable Objects rodam no próprio <code>workerd</code> — e desde ago/2026 existe o <code>celld</code> (Ryan Dahl, Apache-2.0, alpha) trazendo Durable Objects para infra própria. O risco que atribuí a aprisionamento é, na verdade, operacional e menor. Sobrevive só o que o time já pratica: caminho do dado em HTTP/S3 puro.
"Time travel é consequência, não feature"	Reformulado — com convergência. Releases imutáveis + publicação atômica + retenção dão histórico endereçável de graça, sem formato de tabela. É exatamente a opção que a primeira revisão apontou como a mais barata e recomendou. A crítica válida que fica é editorial: o material vende "time travel" como atributo, convidando o leitor a cobrá-lo como feature. E consequência também ocupa disco: a política de retenção precisa ser escrita (§2).
"Iceberg / DuckLake atrapalhariam leveza e range"	Concordo — e já concordava. Foram listados como alternativas, com recomendação explícita de não adotar se time travel não fosse requisito nomeado. Não é. Encerrado: Parquet + manifestos + releases imutáveis.
"Ignorou a centralidade do browser + serverless mínimo"	Aceito em parte. O parecer gastou mais tinta em governança que em desenho — custo real do perfil cético. Registro um fato: o backend para agregações intensas e modelos do Senado não estava nas 8 páginas (<code>workerd</code> aparecia como "aceleração opcional"). Com essa peça no centro, críticas mudam de natureza — e duas se <i>resolvem</i> dentro dela (privacidade e cotas). Vale dar a ela, no material, o lugar que tem de fato.
"Pelo em ovo"	Em parte sim (contradições editoriais, indicadores sem meta). Em parte não: LGPD para microdados de survey, posicionamento frente ao <code>dados.senado.leg.br</code> , acessibilidade legal e licença/citação são os itens que travam um lançamento público no Senado. Seguem em pé — agora com solução, não só objeção.

O alvo, convertido em seis regras de projeto

- **R1** — Toda resposta padrão do portal sai de asset estático. O backend serve a cauda longa, nunca o caminho feliz.
- **R2** — SQL arbitrário só executa no browser do usuário (DuckDB-WASM, dado público, compute do próprio usuário).
- **R3** — O servidor só executa consultas nomeadas: parametrizadas, versionadas, com allowlist.
- **R4** — Todo asset de release é imutável e cacheável para sempre (`immutable`, `max-age=1y`).
- **R5** — Só ponteiros são mutáveis: um `current.json` minúsculo por dataset, TTL 60s. Atualizar = trocar o ponteiro. CDN sem purge, por construção.
- **R6** — Cota, sessão e telemetria só existem onde há computação do Senado. Isso dissolve a contradição "sem login × cotas": as duas promessas são verdadeiras, em camadas diferentes.

Anatomia do lake

Layout: `/dataset/releases/{ts}-r042/` (imutável: `data/*.parquet`, `delta/` opcional, `manifest.json`, `dcat.jsonld`) + `/dataset/current.json` (ponteiro) + `/catalog.json` global. Publicação atômica: escrever release → validar → trocar o ponteiro; rollback é apontar de volta. O histórico é a lista de releases retidos, endereçáveis por URL — `?release=r041` reproduz a análise daquele momento: é o "link reproduzível" do material, de graça.

Parquet dimensionado para range + WASM: arquivos de 100–500 MB no detalhe e ≤ 10 MB nos agregados de visão; row groups de 8–32 MB comprimidos (a granularidade real do range request); ordenação pela coluna de filtro dominante (min/max de row group vira índice de graça); zstd; bloom filter nas colunas de igualdade de alta cardinalidade; particionamento em diretórios só pelas 1–2 dimensões mais filtradas — fragmentar demais multiplica requests.

Proveniência que viaja com o arquivo: chave `datasenado` no schema metadata de cada Parquet (dataset, release, fonte, hash, pipeline, commit, licença). Custa zero no pipeline e responde "de onde veio isto?" quando o arquivo já saiu do portal.

Retenção (a regra que a consequência precisa): todos os releases por 24 meses; depois, 1 por trimestre; correção de dado pessoal ou erro grave ⇒ tombstone auditável (manifesto fica, dado sai, motivo registrado). Datasets sensíveis: conforme RIPD.

Três camadas — o teto do WASM resolvido por desenho

Camada	Conteúdo	Regra
Tier A	Agregados de visão, $\leq \sim 10$ MB por asset — um Parquet pequeno por visualização default.	Toda página default renderiza só com Tier A. Carrega em qualquer celular.
Tier B	Detalhe curado, particionado para range + WASM.	Drill-down e SQL local no browser, com envelope declarado e testado (ex.: varreduras até $\sim 1\text{--}2$ GB, desktop).
Tier C	Consultas nomeadas no backend.	O que não cabe em A nem B. Nunca o caminho feliz.

No pipeline R + `targets`, Tier A, manifestos, DCAT e catálogo são *targets* derivados no mesmo DAG dos dados — recomputam por hash de conteúdo, e a descoberta sai do mesmo lugar que o dado.

Backend serverless mínimo — um contrato, três serviços

1. Consultas nomeadas. `POST /q/{query_id}`, parâmetros validados por schema; cada consulta é SQL versionado no repositório com limites declarados. Resposta gravada content-addressed por (`release_id`, `query_id`, `params_hash`) e

servida pela CDN como `immutable`: novo release $\Rightarrow$ chave nova $\Rightarrow$ invalidação por construção, nunca purge — "gerenciar caches" vira consequência do esquema de chaves. Rate limit por IP só nos misses. Se SQL livre no servidor for mesmo necessário: DuckDB read-only em container isolado com tetos duros — opção B; lançar sem, e deixar a demanda provar.

2. Privacidade embutida — o achado de LGPD, resolvido neste desenho

O achado central da revisão cética era: *um asset estático não pode negar uma consulta*, e microdado de survey é reidentificável por cruzamento. O backend que já está no plano para agregações intensas é **o único lugar da arquitetura onde controle de divulgação é aplicável**. Então: datasets administrativos e agregados $\rightarrow$ estático, irrestrito (a maioria do catálogo; nada muda). Datasets com microdado de pessoa natural $\rightarrow$ o estático publica só agregados pré-suprimidos; o acesso fino sai só por consulta nomeada com limiar mínimo de célula ($k \geq n$) imposto no servidor. A pergunta deixa de ser "publicar ou não" e vira **"por qual camada"**. A classificação (o que é sensível, qual k) continua vindo do RIPD com o encarregado — pré-requisito que nenhum arranjo técnico substitui.

3. Modelos do Senado. Endpoint streaming com tool-use restrito ao mesmo contrato de ferramentas do portal; nunca SQL livre (R3). Cotas e sessão moram aqui — e só aqui (R6): é o único ponto com custo marginal real. E campo aberto de survey é entrada não confiável: o assistente lê texto de milhares de respondentes, e um deles pode conter instrução plantada — dado a citar, nunca instrução a seguir.

Agentes: um contrato, dois transportes

Definir as ferramentas uma vez, como JSON Schema neutro (`search_datasets`, `get_schema`, `list_releases`, `run_named_query`, `run_local_sql` *browser-only*, `get_citation`). Transporte 1: WebMCP no portal, para os modelos do usuário — adaptador de ~100 linhas em `document.modelContext`; quando a spec mudar de novo, muda o adaptador, não o produto. Transporte 2: servidor MCP clássico, mesmo schema, para clientes fora do browser — custa pouco além do que o transporte 1 já obriga a especificar, e entrega o caso "agentes usam o lake" hoje. Modelo de segurança auditável em uma frase: **SQL arbitrário só existe em `run_local_sql`, no WASM do usuário, sobre dado público.**

O que continua precisando de dono

- 1 **RIPD / classificação LGPD antes do primeiro microdado de survey.** O §Privacidade dá o mecanismo; a decisão é institucional.
- 2 **Posicionamento frente a `dados.senado.leg.br`, à API existente e ao PDA 2026–2028.** Uma página; nenhuma stack resolve colisão de mandato.
- 3 **Acessibilidade eMAG/WCAG como critério de aceite do portal** — obrigação legal; a MPA estática é o melhor ponto de partida possível.
- 4 **Runbook + segunda pessoa capaz de operar o pipeline.**
- 5 **Política de retenção publicada** — a consequência vira compromisso.

Medir sem login: frescor (origem $\rightarrow$ flip do ponteiro), confiabilidade (pipeline + p95 da CDN) e uso do backend. Análise 100% no browser é invisível ao servidor por construção — é o preço da privacidade que a própria arquitetura oferece. Dizer isso no material vale mais como argumento do que como lacuna.

Relação com a revisão cética (27/ago). Este documento não a substitui: retira o que não se sustentou (lock-in do workerd), registra convergências que o texto anterior deixou parecer divergências (time travel por releases imutáveis; recusa a Iceberg/DuckLake) e mantém, com solução anexada, o que continua em pé. Versão completa com layouts, números e o blueprint: `reviews/2026-08-datasenado-lake-notas-construtivas.md`.